

gray-matter			
1	2	3	4
8 <pre>const str = fs.readFileSync('example.html', 'utf8'), console.log(matter(str)); Converts a string with front-matter, like this: --- title: Hello slug: home <h1>Hello world!</h1></pre>	7 <pre>const fs = require('fs'); const matter = require('gray-matter'); Add the HTML in the following example to example.html, then add the following code to example.js and run \$ node example (without the \$):</pre> Run this example What does this do?	6 Please consider following this project's author, Jon Schlinkert, and consider starring the project to show your heart: and support. Install with npm: <pre>\$ npm install --save gray-matter</pre> Heads up! Please see the changelog to learn about breaking changes that were made in V3.0.	5 Parse front-matter from a string or file. Fast, reliable and easy to use. Parses YAML front matter by default, but also has support for YAML, JSON, TOML or Coffee Front-Matter, with options to set custom delimiters. Used by metalsmith, assemble, verb and many other projects.
6 Into an object like this: <pre>{ content: '<h1>Hello world!</h1>', data: { title: 'Hello', slug: 'home' } }</pre>	10 Why use gray-matter? simple: main function takes a string and returns an object accurate: better at catching and handling edge cases than front-matter parsers that rely on regex for parsing fast: faster than other front-matter parsers that use regex for parsing flexible: By default, gray-matter is capable of parsing YAML, JSON and JavaScript front-matter. But other engines may be added.	11 extensible: Use custom delimiters, or add support for any language, like TOML, CoffeeScript, or CSON battle-tested: used by assemble, metalsmith, phenomic, verb, generate, update and many others. Rationale Why did we create gray-matter in the first place? We created gray-matter after trying out other libraries that failed to meet our standards and requirements.	12 Some libraries met most of the requirements, but <i>none met all of them</i>. Here are the most important: - Be usable, if not simple - Use a dependable and well-supported library for parsing YAML - Support other languages besides YAML - Support stringifying back to YAML or another language - Don't fail when no content exists - Don't fail when no front matter exists
16 <pre>console.log(matter('---\ntitle: Front Matter\n---\n\nThis is content.', { data: { title: 'Front Matter' } })); Returns: { content: '\nThis is content.', data: { title: 'Front Matter' } }</pre>	15 <pre>const matter = require('gray-matter'); Or with TypeScript import matter = require('gray-matter'); // OR import * as matter from 'gray-matter'; Pass a string and options to gray-matter:</pre>	14 Usage Using Node's require() system: <pre>const matter = require('gray-matter'); Allow custom delimiters, when it's necessary for avoiding delimiter collision. Should return an object with at least these three properties: - data: the parsed YAML front matter, as a JSON object - content: the contents as a string, without the front matter - orig: the "original" content (for debugging)</pre> Usage	13 - Don't use regex for parsing. This is a relatively simple parsing operation, and regex is the slowest and most error-prone way to do it. - Have no problem reading YAML files directly including non-front-matter fenced code blocks that contain examples of YAML front matter. Other parsers fail on this. - Support stringifying back to front-matter, etc.

More about the returned object in the following section. Returned object gray-matter returns a file object with the following properties. Enumerable - file.data {Object}: the object created by parsing front-matter - file.content {String}: the input string, with matter stripped 17	- file.excerpt {String}: an excerpt, if defined on the options - file.empty {String}: when the front-matter is "empty" (either all whitespace, nothing at all, or just comments and no data), the original string is set on this property. See #65 for details regarding use case. - file.isEmpty {Boolean}: true if front-matter is empty. Non-enumerable 18	In addition, the following non-enumerable properties are added to the object to help with debugging. - file.orig {Buffer}: the original input string (or buffer) - file.language {String}: the front-matter language that was parsed. yaml is the default - file.matter {String}: the raw, un-parsed front-matter string - file.stringify {Function}: stringify the file by converting file.data to a string in the 19	given language, wrapping it in delimiters and prepending it to file.content. Run the examples If you'd like to test-drive the examples, first clone gray-matter into my-project (or wherever you want): \$ git clone https://github.com/jonschlinkert/gray-matter my-project 20
Stringify an object to YAML or the specified language, and append it to the given string. stringify <pre>const matter = require('gray-matter'); console.log(matter('---\ntitle: Home\n---\n\nother stuff'));</pre> Example {Object} returns 24	Takes a string or object with content property, extracts and parses front-matter from the string, then returns an object with data, content and other useful properties. Params - input {Object String}: String, or object with content string - options {Object} 23	- excerpt-stringify - excerpt - javascript - json-stringify - json - restore-empty - sections-excerpt - sections - toml - yaml-stringify - yaml 22	CD into my-project and install dependencies: \$ cd my-project && npm install Then run any of the examples to see how gray-matter works: \$ node examples/<example_name> - coffee - excerpt-separator Links to examples 21
By default, only YAML and JSON can be stringified. See the engines section to learn how to stringify other languages. Params - file {String Object}: The content string to append to stringified front-matter, or a file object with file.content string. - data {Object}: Front matter to stringify. - options {Object}: Options to pass to gray-matter and js-yaml. 25	returns {String}: Returns a string created by wrapping stringified yaml with delimiters, and appending that to the given string. Example <pre>console.log(matter.stringify('foo bar baz', {title: 'Home'})); // results in: // --- // title: Home // --- // foo bar baz</pre> 26	.read Synchronously read a file from the file system and parse front matter. Returns the same object as the main function. Params - filepath {String}: file path of the file to read. - options {Object}: Options to pass to gray-matter. - returns {Object}: Returns an object with data and content 27	Example <pre>const file = matter.read('./content/blog-post.md');</pre> .test Returns true if the given string has front matter. Params - string {String} - options {Object} 28
// returns the first 4 lines of the contents function firstFourLines(file, options) { file.excerpt = file.content.split('\n').slice(0, 4).join(' '); } const file = matter({ }, 32	content: 'This is an excerpt.\n---\n\nthis is content', data: { foo: 'bar' }, excerpt: 'This is an excerpt.\n' } You can also set excerpt to a function. This function uses the 'file' and 'options' that were initially passed to gray-matter as parameters, so you can control how the excerpt is extracted from the content. Example 31	If set to excerpt: true, it will look for the frontmatter delimiter, --- by default and grab everything leading up to it. Example <pre>const str = '---\nfoo: bar\n---\n\nthis is an excerpt.\n---\n\nthis is content'; const file = matter(str, { excerpt: true });</pre> Results in: 30	returns {Boolean}: True if front matter exists. Options options.excerpt Type: Boolean Function Default: undefined Extract an excerpt that directly follows front-matter, or is the first thing in the string if no front-matter exists. 29

<pre>'foo: bar', '----', 'Only this', 'will be', 'in the', 'excerpt', 'but not this...'].join('\n'), {excerpt: firstFourLines});</pre> Results in:	<pre>{ content: 'Only this\nwill be\nin the\nexcerpt\nbut not this...', data: { foo: 'bar' }, excerpt: 'Only this will be in the excerpt' }</pre> options.excerpt_separator Type: String	Default: undefined Define a custom separator to use for excerpts. <pre>console.log(matter(string, {excerpt_separator: '<!-- end -->' }));</pre> Example The following HTML string: <pre>--- title: Blog</pre>	<pre>--- My awesome blog. <!-- end --> <h1>Hello world</h1></pre> Results in: <pre>{ data: { title: 'Blog'}, excerpt: 'My awesome blog.',</pre>
<pre> } stringify to TOML'); throw new Error('cannot stringify:'); { } it's just an example stringifier might exist, this // not supported (a TOML is to let users know stringifying // example of throwing an error parse: toml.parse.bind(toml),</pre>	<pre> engines: { toml: toml.parse.bind(toml), } } /** * Or as an object */ const file = matter(str, { engines: { toml: } });</pre>	<pre>const toml = require('toml'); /** * defined as a function */ const file = matter(str, {</pre> Engine format Engines may either be an object with parse and (optionally) stringify methods, or a function that will be used for parsing only. Examples	<pre> } content: 'My awesome blog. \n<!-- end -->\n<h1>Hello world</h1>' }</pre> Define custom engines for parsing and/or stringifying front-matter. Type: Object of engines Default: JSON, YAML and JavaScript are already handled by default.
<pre> } } }(); console.log(file);</pre> options.language Type: String Default: yaml Define the engine to use for parsing front-matter.	<pre>console.log(matter(string, {language: 'toml'}));</pre> Example The following HTML string: <pre>--- title = "TOML" description = "Front matter" categories = "front matter toml" ---</pre> This is content	Results in: <pre>{ content: 'This is content', excerpt: '', data: { title: 'TOML', description: 'Front matter', categories: 'front matter toml' } }</pre> Dynamic language detection Instead of defining the language on the options, gray-matter will automatically detect	the language defined after the first delimiter and select the correct engine to use for parsing. <pre>---toml title = "TOML" description = "Front matter" categories = "front matter toml" ---</pre> This is content
options.parsers Deprecated, please use options.engines instead. About Contributing Pull requests and stars are always welcome. an issue, please create Running Tests	Deprecated options Deprecated, please use options.language instead. options.delims Deprecated, please use options.delimiters instead.	<pre>// or an array (open/close matter.read('file.md', {delims: ['~~~', '~~~']}); would parse: ~~~ title: Home ~~~ This is the {{title}} page.</pre>	options.delimiters Type: String Default: --- Open and close delimiters can be passed in as an array of strings. Example: <pre>matter.read('file.md', {delims: ['~~~', '~~~']});</pre>

Running and reviewing unit tests is a great way to get familiarized with a library and its API. You can install dependencies and run tests with the following command: <pre>\$ npm install && npm test</pre> Building docs <i>(This project's readme.md is generated by verb, please don't edit the readme directly. Any changes to the readme must be made in the .verb.md readme template.)</i>	To generate the readme, run the following command: <pre>\$ npm install -g verbose/verb#dev verb-generate-readme && verb</pre> Related projects You might also be interested in these projects: assemble: Get the rocks out of your socks! Assemble makes you fast at creating web projects... more homepage	metalsmith: An extremely simple, pluggable static site generator. homepage verb: Documentation generator for GitHub projects. Verb is extremely powerful, easy to use, and is used... more homepage gray-matter-loader: A webpack loader for gray-matter. homepage Contributors <table><tr><th>Commits</th><th>Contributor</th></tr><tr><td>174</td><td>jonschlinkert</td></tr></table>	Commits	Contributor	174	jonschlinkert	<table><tr><th>Commits</th><th>Contributor</th></tr><tr><td>7</td><td>RobLoach</td></tr><tr><td>5</td><td>heymind</td></tr><tr><td>4</td><td>doowb</td></tr><tr><td>3</td><td>aljopro</td></tr><tr><td>2</td><td>reccanti</td></tr><tr><td>2</td><td>onokumus</td></tr><tr><td>2</td><td>moozyyk</td></tr><tr><td>1</td><td>Ajedi32</td></tr></table>	Commits	Contributor	7	RobLoach	5	heymind	4	doowb	3	aljopro	2	reccanti	2	onokumus	2	moozyyk	1	Ajedi32
Commits	Contributor																								
174	jonschlinkert																								
Commits	Contributor																								
7	RobLoach																								
5	heymind																								
4	doowb																								
3	aljopro																								
2	reccanti																								
2	onokumus																								
2	moozyyk																								
1	Ajedi32																								
		Author Jon Schlinkert LinkedIn Profile GitHub Profile Commits Contributor caesar 1 ianstormtaylor 1 gm3ster 1 zachwhaley 1																							
		License Copyright © 2018, Jon Schlinkert. Released under the MIT License. This file was generated by verb-generate-readme, v0.6.0, on April 01, 2018. Twitter Profile																							